



디버깅, 라이브러리

Makefile 복습

- 최종 실행파일 : travel
- 기본 파일 : main.c, kor.c, usa.c, fra.c
- 위의 소스 파일을 기준으로 Makefile을 완성하시오.
 - 1단계 : 기본 Makefile
 - 2단계 : 변수 활용
 - 3단계 : 단순화

GDB

- GDB (GNU DeBugger)
 - GNU 프로젝트의 디버거
 - C, C++, Objective-C, Modular 2, Ada 등의 언어 지원
 - 기능
 - 프로그램을 시작한다.
 - 프로그램이 특정 조건을 만족할 때 정지시킨다.
 - 프로그램이 정지했을 때 어떤 일이 벌어졌는지 조사한다.
 - 프로그램 일부를 수정하여 발견한 버그를 제거했을 때의 실행 결과를 미리 알아볼 수 있게 한다.
 - 요구사항
 - Gcc 컴파일 시에 "-g" 옵션을 사용하여 디버깅 정보를 오브젝트 코드에 삽입

디버깅

- GCC 컴파일 시 -g 옵션을 사용하여 디버깅 정보를 포함하고, gdb 디버거를 사용하여 프로그램을 실행한 후 중단점 설정, 변수 값 확인, 코드 실행 등을 통해 버그를 찾고 수정할 수 있다.
- GDB 설치 및 실행
 - GDB 설치: 터미널에서 `sudo apt update && sudo apt install gdb`
 - 컴파일: 디버깅을 위한 정보를 포함하여 C 코드를 컴파일

-g 옵션과 디버깅의 개념

- 디버깅(Debugging)
 - 프로그램에서 발생한 오류의 원인을 찾아 수정하는 과정

오류 종류	증상	예
문법 오류	컴파일이 되지 않음	세미콜론 누락, 잘못된 변수명
논리 오류	실행되지만 결과가 잘못됨	합계 계산에서 마지막 숫자 누락
실행 중 오류	실행 도중 비정상 종료됨	잘못된 포인터 접근

- 컴파일은 되지만 결과가 틀리거나 실행 중 종료되는 프로그램을 GDB로 분석
- -g 자체가 디버거를 실행하거나 오류를 자동으로 고치는 것은 아닙니다.

GDB

- `gcc -g main.c -o main`
- `-g` 옵션은 디버깅 심볼을 포함시키므로, 소스 코드 라인별 디버깅이 가능
- GDB 실행: 컴파일된 실행 파일을 `gdb`로 실행
- `gdb ./main`

GDB 도움말

- Files 도움말

- 작업 디렉토리 설정/검색, 실행 파일 지정 등의 파일과 관련된 명령어 모음
- 잘 사용하는 명령어
 - **cd**: 디버거와 디버깅중인 프로그램에 대한 작업 디렉토리를 변경함.
 - **directory**: 소스 파일에 대한 탐색경로를 추가함.
 - **file**: 디버깅할 파일을 지정함.
 - **list**: 지정된 함수 또는 줄을 보여줌.
 - **path**: 목적 파일에 대한 탐색경로를 추가함.
 - **pwd**: 현재의 작업 디렉토리를 보여줌.
 - **search**: 정규 수식을 가장 최근에 보여진 줄 다음부터 탐색함.

GDB 도움말

- Running 도움말

- 단계별 실행과 관련된 명령어 모음
- 잘 사용하는 명령어

- run: 디버깅할 프로그램을 시작함. run 명령어의 프로그램 인자는 디버깅할 프로그램의 프로그램 인자와 같은 역할을 함.
- continue: 디버깅중인 프로그램의 진행을 계속함.
- kill: 디버깅중인 프로그램의 실행을 정지함.
- step: 다음 줄에 도달할 때까지 프로그램을 진행시킴. 인자 N은 이를 N번 반복함을 의미함. 함수 호출을 하게 되면 해당 함수로 제어가 넘어감.
- next: step 명령어와 비슷하나, 함수 호출을 하나의 단위로 보아 함수 안을 디버깅할 수 없음. 올바르게 동작하는 함수의 경우 더 이상 디버깅이 필요 없을 때 사용하면 편리함.

GDB 도움말

- Breakpoints 도움말

- 프로그램을 특정 지점에서 정지하게 하기 위한 명령어 모음
- 잘 사용하는 명령어
 - break: 소스 내의 특정 줄이나 특정 함수에 대한 정지점(breakpoint)을 설정할 때 사용함. break 명령어의 인자로 줄 번호가 지정되면 해당 줄 번호에 정지점이 설정되며, 인자로 함수 이름이 지정되면 함수의 시작 부분에 정지점이 설정됨.
 - watch: 변수 등에 대한 경계점/watchpoint)을 설정할 때 사용됨. 지정된 경계점의 값이 변경될 경우 프로그램 실행을 정지함.
 - clear: 특정 줄이나 특정 함수에 설정된 정지점을 해제할 때 사용함.
 - delete: 정지점 등을 해제할 때 사용함. clear와 달리 정지점의 번호를 인자로 사용함.

GDB 도움말

- Data 도움말

- 변수, 메모리의 정보를 검색하고 설정하는 명령어 모음
- 잘 사용하는 명령어
 - display: 프로그램 실행이 정지될 때마다 특정 수식의 값을 자동적으로 보여줌. 디버깅할 때 특정 변수의 값이 변동하는 것을 관찰할 때 편리함.
 - delete display: display 명령어에 의하여 지정된 수식을 제거함.
 - print: 해당 수식의 값을 보여줌. display 명령어와 달리 일회성의 성격을 띠م.
 - set variable: 특정 변수의 값을 설정하는데 사용함. 디버깅하면서 특정 변수의 값을 변화시켜 가면서 프로그램의 변화를 살펴보는데 유용함.
 - whatis: 해당 수식의 데이터 형을 보여줌.

GDB 도움말

- Info 도움말

- 디버깅 중인 프로그램에 대한 여러 정보를 보여주는 명령어 모음
- 잘 사용하는 명령어
 - info args: 프로그램 인자에 대한 정보를 보여줌.
 - info breakpoints: 지정된 정지점에 대한 정보를 보여줌.
 - info display: display 명령어에 의하여 지정된 수식 정보를 보여줌.
 - info files: 디버깅 중인 프로그램에 대한 정보를 보여줌.
 - info locals: 현재 스택 프레임내 지역 변수에 대한 정보를 보여줌.
 - info types: 현재 정의된 데이터형에 대한 정보를 보여줌.

GDB 기본 명령어

- break 또는 b: 특정 라인에 중단점(breakpoint)을 설정
 - 예: break main (main 함수 시작에 중단점 설정)
- run 또는 r: 프로그램을 실행
- next 또는 n: 다음 라인으로 한 줄씩 실행
(함수 호출 시 함수 안으로 들어가지 않음).
- step 또는 s: 다음 라인으로 실행하며, 함수 호출 시 함수 안으로 들어간다.
- print 또는 p: 변수의 현재 값을 출력
 - 예: print my_variable
- continue 또는 c: 중단점까지 프로그램 실행을 계속
- list 또는 l: 현재 위치의 소스 코드를 보여준다.
- quit 또는 q: GDB를 종료합니다.

디버깅 예제1

- BMI 계산기를 만들어 보자
 - BMI = 몸무게 / (키 * 키)
- bmi_cal.c
- 기본 컴파일 해서 확인
- gcc bmi_cal.c
- ./a.out

```
1 #include<stdio.h>
2
3 double bmi_calculate(double h, double w);
4 int main(void){
5     double height, weight, bmi;
6     printf("BMI 계산하기\n");
7     printf("키와 몸무게를 입력해 주세요 : ");
8     scanf("%lf %lf", &height, &weight);
9     bmi = bmi_calculate(height, weight);
10    printf("당신의 BMI : %lf\n", bmi);
11    return 0;
12 }
13
14 double bmi_calculate(double h, double w){
15     double BMI;
16     BMI = w / (h * h);
17     return BMI;
18 }
```

디버깅 해보기

- `gcc -g -o bmi bmi_cal.c`
- `gdb bmi`

디버깅 예제2

```
#include <stdio.h>

int main(void)
{
    int n = 5;
    int sum = 0;
    int i;

    for (i = 1; i < n; i++) {
        sum += i;
    }

    printf("1부터 %d까지의 합: %d\n", n, sum);

    return 0;
}
```

디버깅 예제3

```
#include <stdio.h>

double average(int a, int b)
{
    double result = (a + b) / 2;
    return result;
}

int main(void)
{
    int x = 5;
    int y = 2;
    double answer;

    answer = average(x, y);

    printf("평균: %.1f\n", answer);

    return 0;
}
```


디버깅 예제4

```
#include <stdio.h>

void change_score(int *p)
{
    *p = 100;
}

void process_score(int *p)
{
    change_score(p);
}

int main(void)
{
    int score = 70;
    int *ptr = NULL;

    process_score(ptr);

    printf("점수: %d\n", score);

    return 0;
}
```

기본 라이브러리

POSIX 표준 라이브러리 함수

- POSIX (Portable Operating System Interface) 표준
 - IEEE 에서 1988년에 UNIX 라이브러리를 위한 POSIX.1 발표
 - 1998년 The Open Group, IEEE POSIX, ISO/IEC 연합기술운영회의 회원들로 구성된 Austin 그룹 결성
 - 2001, 2002년 IEEE 표준 1003.1-2001 제정
 - 기타 표준 구성
 - POSIX.2 - 표준 셸과 유틸리티 인터페이스
 - POSIX.4 - 스레드 관리
 - POSIX.5 - Ada 언어를 위한 바인딩
 - POSIX.6 - 보안 기능

표준 C 라이브러리 함수들

- C 언어 표준 라이브러리

헤더 파일	함수 기능	표준 함수
stdio.h	표준입출력	printf(); scanf(); putchar(); getchar(); ...
	화일입출력	fopen(); fclose(); fprintf(); ...
stdlib.h	수치변환	atoi(); itoa(); ...
	난수발생	rand(); srand(); ...
	탐색 및 정렬	bsearch(); qsort();...
ctype.h	문자변환	tolower(); toupper(); ...
	문자판별	isalpha(); isdigit(); isupper(); ...
String.h	문자열처리	strcpy(); strlen(); strcmp(); ...
	동적메모리관리	memcpy(); memset(); memchr(); ...
Math.h	수학함수	sin(); cos(); sqrt(); ...
Search.h	탐색	lsearch(); hsearch(); tsearch(); ...
Time.h	시간관련함수	time(); difftime(); ctime(); ...

표준 C 라이브러리 함수

- 표준 C 라이브러리 함수 사용 예
 - 표준 C 라이브러리 함수 예제 프로그램

```
/*
 * 기본 라이브러리 함수
 * 파일이름: library.c
 */
#include <stdio.h>
#include <math.h>
#include <ctype.h>

int main(void)
{
    int i=4;
    char c='A';

    printf("sqrt(%d) = %f\n", i, sqrt(i));
    printf("tolower(%c) = %c\n", c, tolower(c));
}
```

- 프로그램 실행 결과

```
[cprog2@seps5 ch7]$ gcc -o library library.c -lm
[cprog2@seps5 ch7]$ ./library
sqrt(4) = 2.000000
tolower(A) = a
[cprog2@seps5 ch7]$
```

프로그램 인터페이스

- main() 함수의 매개 변수
 - Main() 함수의 기본형

```
main(int argc, char *argv[])  
{  
    프로그램 명령들  
}
```

- 사용 예

```
$ main a b c
```

main	→	argv[0]
a	→	argv[1]
b	→	argv[2]
c	→	argv[3]

main() 함수의 매개변수

- 매개변수 사용 예
 - 매개변수 예제 프로그램

```
/*
 * 기본 라이브러리 함수
 * 파일이름: main.c
 */

#include <stdio.h>
int main(int argc, char *argv[])
{
    int i;
    for (i=0; i<argc; i++) printf("%s ", argv[i]);
}
```

- 매개변수 예제 프로그램 실행 결과

```
[cprog2@seps1 cprog2]$ gcc -o main main.c
[cprog2@seps1 cprog2]$ ./main string 10
./main string 10
[cprog2@seps1 cprog2]$
```

main() 함수의 매개변수

* 매개변수 예제 프로그램 실행 예

```
[dhju@localhost ~]$ cat main.c
#include <stdio.h>
main(int argc, char *argv[])
{
    int i;
    printf("argc --> %d \n", argc);
    for(i=0; i<argc ; i++)
    {
        printf("argv[%d] --> %s \n", i, argv[i]);
    }
}
```

* 매개변수 예제 프로그램 실행 결과

```
[dhju@localhost ~]$ gcc main.c -o main
[dhju@localhost ~]$ ./main string 10
argc --> 3
argv[0] --> ./main
argv[1] --> string
argv[2] --> 10
[dhju@localhost ~]$
```


환경변수 처리

- 환경변수 처리
 - main() 함수의 확장형

```
main(int argc, char *argv[], char *envp[])
{
    프로그램 명령들
}
```

- 많이 사용하는 환경변수

환경변수	설명
USER	로그인한 사용자 이름
LOGNAME	프로세스와 관련된 로그인 사용자 이름
HOME	사용자의 로그인 디렉토리
LANG	LC_ALL 등이 지정되지 않았을 때의 로케일 이름
LC_ALL	우선적으로 지정되는 로케일 이름. LC_COLLATE, LC_CTYPE, LC_MESSAGES, LC_MONETARY, LC_NUMERIC, LC_TIME 등을 포함한다.
PATH	실행 파일을 찾는 디렉토리들의 목록
PWD	현재 작업 디렉토리
SHELL	사용자의 로그인 쉘 파일 이름
TERM	출력 터미널 타입
TMPDIR	임시 디렉토리 경로
LD_LIBRARY_PATH	동적 로딩/링킹을 위한 라이브러리 경로

환경변수 처리

- 환경변수 처리 사용 예
 - 환경변수 처리 예제 프로그램

```
/*
 * 기본 라이브러리 함수
 * 파일이름: envp.c
 */
#include <stdio.h>
int main(int argc, char *argv[], char *envp[])
{
    int i;
    for (i=0; envp[i]; i++)
        printf("%s\n", envp[i]);
}
```

- 환경변수 처리 예제 프로그램 실행 결과

```
[cprog2@seps1 cprog2]$ gcc -o envp envp.c
[cprog2@seps1 cprog2]$ ./envp
HOSTNAME=....
TERM=xterm
....
_=./envp
[cprog2@seps1 cprog2]$
```